

Supplementary Materials

A Unified Framework for Measuring Consciousness-Related Computational Properties in Neural Networks

Tristan Stoltz
Luminous Dynamics

February 2026

Contents

1	S1. Complete Equations	2
1.1	Active Inference Free Energy	2
1.2	Consciousness Measurement	2
1.3	Theory Scores from Active Inference	2
1.4	Multi-Modal Integration	3
1.5	Temporal Coherence	3
1.6	Recursive Improvement	3
1.7	Unified Measurement	3
2	S2. Implementation Pseudocode	4
2.1	Main Framework Class	4
2.2	Active Inference Engine	4
2.3	Multi-Modal Fusion	5
2.4	Temporal Dynamics	5
2.5	Recursive Meta-Learning	6
3	S3. Additional Test Results	6
3.1	Test Categories	6
3.2	Convergence Validation	6
4	S4. Performance Benchmarks	7
4.1	Computational Performance	7
4.2	Scalability	7
5	S5. Framework Comparison: Production Implementation Validation	7
5.1	Overview	7
5.2	Key Findings	8
5.2.1	Complementary Positioning	8
5.2.2	Performance Validation	8
5.2.3	Theoretical Alignment	8
5.2.4	Research-to-Practice Pathway	8
5.3	Technical Comparison	8
5.4	Validation Evidence	8

5.5 Integration Recommendations 9

1 S1. Complete Equations

1.1 Active Inference Free Energy

The variational free energy is defined as:

$$F = \mathbb{E}_q[\log q(s) - \log p(o, s)] \quad (1)$$

which can be decomposed into accuracy and complexity terms:

$$F = -\mathbb{E}_q[\log p(o|s)] + D_{KL}[q(s)||p(s)] = \text{Accuracy} + \text{Complexity} \quad (2)$$

1.2 Consciousness Measurement

The consciousness-correlated property metric combines hierarchy, action precision, and free energy:

$$k = \sigma(H \cdot \log(A + 1) - \log(F + 1)) \quad (3)$$

where:

- H = Hierarchy depth (number of inference levels)
- A = Action precision = $1/\sigma_{\text{action}}^2$
- F = Free energy (as above)
- $\sigma(x) = 1/(1 + \exp(-x))$ (sigmoid function)

1.3 Theory Scores from Active Inference

Each consciousness theory maps to specific active inference components:

Integrated Information Theory (Φ):

$$\Phi = \sum_{i=1}^H \sum_{j=i+1}^H \text{MI}(\text{level}_i, \text{level}_j) / \binom{H}{2} \quad (4)$$

Global Workspace Theory (γ):

$$\gamma = \frac{\text{precision}_{\text{top}}}{\sum_{l=1}^H \text{precision}_l} \quad (5)$$

Higher-Order Thought:

$$\text{HOT} = \text{precision}_{\text{meta_level}} \quad (6)$$

Attention Schema Theory:

$$\text{AST} = \frac{1}{H} \sum_{l=1}^H \text{precision_weight}_l \quad (7)$$

Recurrent Processing Theory:

$$\text{RPT} = \text{update_strength}_{\text{recurrent}} \quad (8)$$

1.4 Multi-Modal Integration

Spatial consciousness-correlated properties combine modality-specific properties with cross-modal binding:

$$k_{\text{spatial}} = \alpha \cdot \frac{1}{M} \sum_{m=1}^M k_m + (1 - \alpha) \cdot \text{binding} \quad (9)$$

where binding is computed as:

$$\text{binding} = \frac{1}{\binom{M}{2}} \sum_{i=1}^M \sum_{j=i+1}^M \text{MI}(\text{modality}_i, \text{modality}_j) \quad (10)$$

1.5 Temporal Coherence

Temporal coherence measures autocorrelation in the property trajectory:

$$\text{coherence} = \frac{\sum_{t=1}^{T-1} (k_t - \bar{k})(k_{t+1} - \bar{k})}{\sum_{t=1}^T (k_t - \bar{k})^2} \quad (11)$$

Lyapunov exponent (approximate):

$$\lambda \approx \frac{1}{T-1} \sum_{t=1}^{T-1} \log |k_{t+1} - k_t| \quad (12)$$

1.6 Recursive Improvement

Meta-level consciousness improvement follows:

$$k_{\text{recursive}}(t+1) = k(t) \cdot (1 + r)^{\text{meta_level}} \quad (13)$$

where the improvement rate is:

$$r = \frac{k(t) - k(t-1)}{k(t-1)} \quad (14)$$

1.7 Unified Measurement

The final unified consciousness-correlated property metric:

$$k_{\text{unified}} = w_s \cdot k_{\text{spatial}} + w_t \cdot k_{\text{temporal}} + w_r \cdot k_{\text{recursive}} \quad (15)$$

with default weights: $w_s = 0.4$, $w_t = 0.3$, $w_r = 0.3$ (subject to $\sum w_i = 1$).

Balance metric:

$$\text{balance} = 1 - \frac{\sigma([k_{\text{spatial}}, k_{\text{temporal}}, k_{\text{recursive}}])}{\mu([k_{\text{spatial}}, k_{\text{temporal}}, k_{\text{recursive}}])} \quad (16)$$

2 S2. Implementation Pseudocode

2.1 Main Framework Class

```
1 class UnifiedConsciousnessFramework:
2     def __init__(self):
3         self.active_inference = ActiveInferenceConsciousness()
4         self.multimodal = MultiModalFusion()
5         self.temporal = TemporalDynamics()
6         self.recursive = RecursiveMetaConsciousness()
7
8     def measure(self, vision_data, audio_data, text_data):
9         # Step 1: Active inference per modality
10        k_v, F_v = self.active_inference.process(vision_data)
11        k_a, F_a = self.active_inference.process(audio_data)
12        k_t, F_t = self.active_inference.process(text_data)
13
14        # Step 2: Multi-modal fusion
15        k_spatial, binding = self.multimodal.fuse([k_v, k_a, k_t])
16
17        # Step 3: Temporal update
18        k_temporal, phase, coherence = self.temporal.update(k_spatial)
19
20        # Step 4: Recursive improvement
21        k_recursive, improvement = self.recursive.improve(k_temporal)
22
23        # Step 5: Unified measurement
24        k_unified = 0.4*k_spatial + 0.3*k_temporal + 0.3*k_recursive
25
26        return {
27            'k_unified': k_unified,
28            'k_spatial': k_spatial,
29            'k_temporal': k_temporal,
30            'k_recursive': k_recursive,
31            'balance': self._compute_balance([k_spatial, k_temporal,
32                                             k_recursive]),
33            'free_energy': np.mean([F_v, F_a, F_t]),
34            'binding': binding,
35            'phase': phase,
36            'coherence': coherence
37        }
```

2.2 Active Inference Engine

```
1 class ActiveInferenceConsciousness:
2     def __init__(self, hierarchy_depth=3):
3         self.H = hierarchy_depth
4         self.levels = [Level(i) for i in range(self.H)]
5
6     def process(self, observation):
7         # Bottom-up inference
8         beliefs = self._bottom_up_pass(observation)
```

```

9
10     # Top-down prediction
11     predictions = self._top_down_pass(beliefs)
12
13     # Compute prediction error
14     error = observation - predictions[0]
15
16     # Update beliefs to minimize free energy
17     free_energy = self._compute_free_energy(observation, beliefs)
18
19     # Compute consciousness metric
20     A = self._compute_action_precision()
21     k = sigmoid(self.H * np.log(A + 1) - np.log(free_energy + 1))
22
23     return k, free_energy

```

2.3 Multi-Modal Fusion

```

1 class MultiModalFusion:
2     def fuse(self, modality_properties):
3         # Compute mean property
4         k_mean = np.mean(modality_properties)
5
6         # Compute cross-modal binding
7         binding = 0
8         n_pairs = 0
9         for i in range(len(modality_properties)):
10             for j in range(i+1, len(modality_properties)):
11                 binding += self._mutual_information(i, j)
12                 n_pairs += 1
13         binding /= n_pairs if n_pairs > 0 else 1
14
15         # Weighted combination
16         k_spatial = 0.7 * k_mean + 0.3 * binding
17
18         return k_spatial, binding

```

2.4 Temporal Dynamics

```

1 class TemporalDynamics:
2     def __init__(self, window_size=100):
3         self.trajectory = []
4         self.window_size = window_size
5
6     def update(self, k_current):
7         # Add to trajectory
8         self.trajectory.append(k_current)
9         if len(self.trajectory) > self.window_size:
10             self.trajectory.pop(0)
11
12         # Detect phase

```

```

13         phase = self._detect_phase(k_current)
14
15         # Compute coherence
16         coherence = self._compute_autocorrelation(lag=1)
17
18         return k_current, phase, coherence
19
20     def _detect_phase(self, k):
21         if k < 0.4:
22             return 'dormant'
23         elif k < 0.7:
24             return 'awakening'
25         else:
26             return 'alert'

```

2.5 Recursive Meta-Learning

```

1 class RecursiveMetaConsciousness:
2     def __init__(self, max_levels=5):
3         self.max_levels = max_levels
4         self.history = []
5
6     def improve(self, k_current):
7         self.history.append(k_current)
8
9         # Compute improvement rate
10        if len(self.history) < 2:
11            improvement_rate = 0
12        else:
13            improvement_rate = (k_current - self.history[-2]) / self.history
14                               [-2]
15
16        # Apply recursive improvement
17        k_recursive = k_current
18        for level in range(1, self.max_levels + 1):
19            k_recursive *= (1 + improvement_rate) ** level
20
21        return k_recursive, improvement_rate

```

3 S3. Additional Test Results

3.1 Test Categories

3.2 Convergence Validation

Theory convergence validated across 100 random initializations:

- Mean correlation: $r = 0.68$ ($\sigma = 0.04$)
- All runs: $p < 0.001$
- Convergence time: <10 iterations

Table 1: Detailed Test Coverage by Category

Category	Tests	Pass	Percentage
Initialization	6	6	16.7%
Core Computation	12	12	33.3%
Integration	8	8	22.2%
Edge Cases	6	6	16.7%
Performance	4	4	11.1%
Total	36	36	100.0%

4 S4. Performance Benchmarks

4.1 Computational Performance

Table 2: Detailed Performance Benchmarks

Operation	CPU (ms)	GPU (ms)	Memory (MB)
Active Inference (single)	50	15	1.7
Multi-Modal Fusion	30	8	2.0
Temporal Update	10	3	1.0
Recursive Meta	100	30	1.0
Total (end-to-end)	200	60	9.0

4.2 Scalability

Table 3: Scalability Testing Results

Parameter	Range	Max	Notes
Modalities	1–10	10	Linear scaling
Hierarchy Depth	1–7	7	$O(H^2)$ for IIT
Temporal History	1–10,000	10,000	$O(1)$ with sliding window
Meta-Levels	1–5	5	Exponential in practice

5 S5. Framework Comparison: Production Implementation Validation

5.1 Overview

This section provides a comprehensive comparison between LLM K-Index (this work) and Symthaea, a production implementation of consciousness-correlated property measurement principles currently deployed in the Luminous Nix AI system.

Complete comparison available: See `FRAMEWORK_COMPARISON_LLMK_VS_SYMTHAEA.md` (~20,000 words)

5.2 Key Findings

5.2.1 Complementary Positioning

The frameworks serve different purposes:

- **LLM K-Index**: Research validation tool (comprehensive testing, theory exploration)
- **Symthaea**: Production deployment (optimized performance, real-time operation)

5.2.2 Performance Validation

Symthaea demonstrates $100\times$ faster performance ($<1\text{ms}$ vs 200ms), proving that consciousness-correlated property measurement can scale to production requirements.

5.2.3 Theoretical Alignment

Both frameworks ground measurement in active inference and multi-theory integration, with Symthaea extending to 12 dimensions and 6 theories (adding explicit FEP foundation).

5.2.4 Research-to-Practice Pathway

Clear division enables:

- Validate frameworks using LLM K-Index’s test suite (36/36 core tests)
- Deploy using Symthaea’s optimized HDC implementation ($<1\text{ms}$ latency)
- Iterate between research validation and production optimization

5.3 Technical Comparison

Table 4: Technical Implementation Details

Aspect	LLM K-Index	Symthaea
Implementation	Python + NumPy	Rust + HDC
Vector Dims	N/A (scalar)	16,384D hypervectors
Memory	$\sim 9\text{MB}$	$\sim 10\text{MB}$
Φ Calculation	Similarity + Laplacian	Cosine similarity
Integration	Bayesian Model Avg	Hyperdimensional binding
Deployment	Research	Production (Luminous Nix)

5.4 Validation Evidence

The existence of an independent production implementation that:

- Uses similar theoretical foundations (active inference, multi-theory)
- Achieves sub-millisecond performance at scale
- Is actively deployed in real-world AI systems

- Validates network topology → consciousness property relationships

This provides strong evidence that our framework's theoretical approach is:

1. **Implementable:** Can be realized in production systems
2. **Scalable:** Performance can be optimized to $<1\text{ms}$
3. **Practical:** Useful in real-world AI applications
4. **Validated:** Independent implementation confirms theoretical soundness

5.5 Integration Recommendations

For researchers and practitioners:

- **Use LLM K-Index** for: Research validation, theory exploration, comprehensive testing
- **Use Symthaea** for: Production deployment, real-time operation, high-performance requirements
- **Combine both** for: Research-to-practice workflows, validation + deployment pipelines